

Recherche Sémantique & RAG

Retrieval-Augmented Generation & Multimodal RAG

Carl HATOUM Données Spécifiques — 2026

Plan du Cours

Pilier 1 : Recherche Sémantique

- De la lexique au sens.
- Création d'embeddings et bases de données vectorielles.



Pilier 2 : RAG (Retrieval- Augmented Generation)

- Pipelines de recherche augmentée
- Reranking
- Génération fidèle.



Pilier 3 : Multimodal & Avancé

- Étude de cas
- Modèles CLIP
- Intégration d'images/tableaux.



1

Recherche Sémantique

De la lexique au sens

De la Recherche Lexicale à la Sémantique

Recherche Lexicale

- Correspondance exacte de mots-clés
- TF-IDF, BM25, Elasticsearch
- chat ≠ félin (synonymes ignorés)
- Rapide et transparent
- Problème : requête ≠ vocabulary du doc

Recherche Sémantique

- Représentation vectorielle dense
- Exemples de modèles : Sentence-BERT, E5, nomic-embed
- chat ≈ félin (similarité = 0.92)
- Gère paraphrases et synonymes
- Comprend l'intention de la requête

Critère	Lexicale	Sémantique
Méthode	TF-IDF, BM25	SBERT, E5, nomic
Représentation	Vecteur creux ~100K dims	Vecteur dense 384–1024 dims
Synonymes	✗ Non gérés	✓ chat ≈ félin (cos=0.92)
Paraphrases	✗ Non gérées	✓ Gérées
Vitesse	< 1ms ⚡	10–50ms
Transparence	✓ Totale	⚠ Boîte noire
Usage idéal	Codes, termes exacts	Questions naturelles
Recommandé	Hybrid (BM25+Dense) = meilleur des deux	

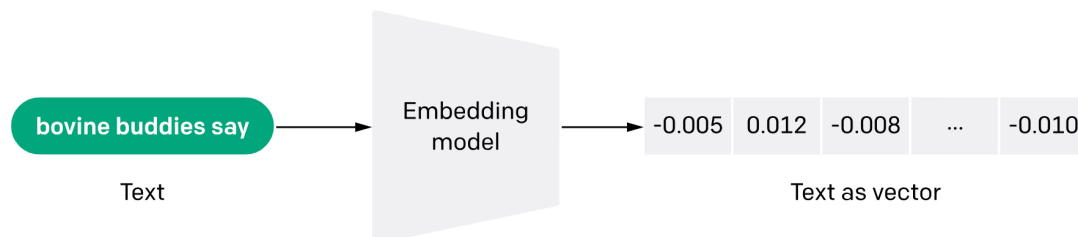
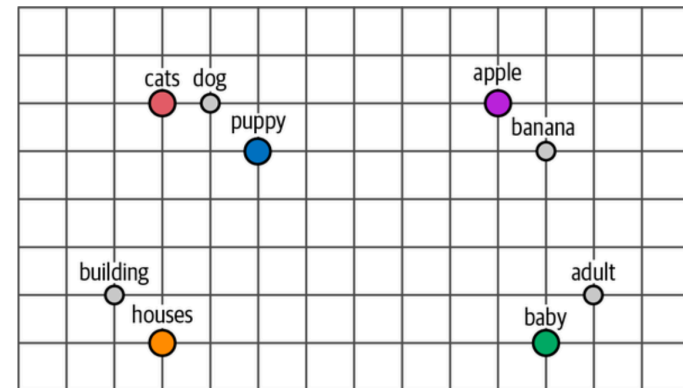
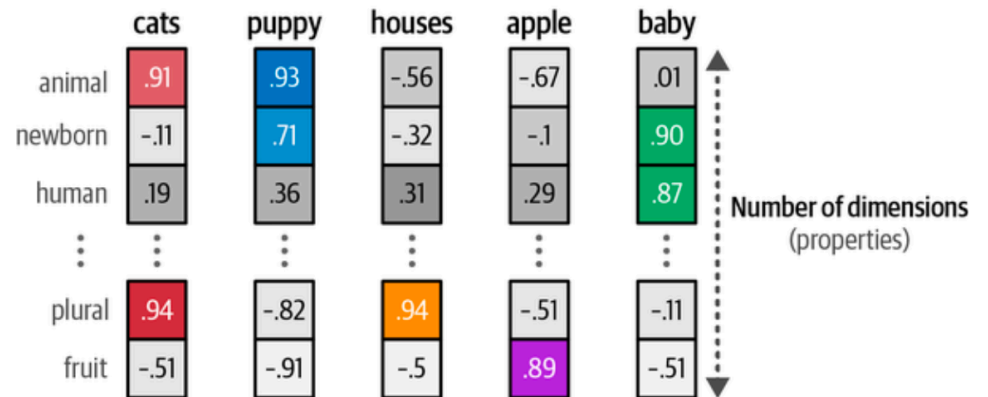
Les Embeddings — Vecteurs Denses de Sens

Qu'est-ce qu'un embedding ?

- Représentation numérique continue d'un texte
- Vecteur de 384 à 1536 dimensions
- Distance cosinus mesure la similarité sémantique
- Textes similaires = voisins dans l'espace
- Appris par réseau de neurones sur corpus

Exemple concret

- "Un chat assis" → [0.234, -0.891, 0.156, ...]
- "Un félin repose" → [0.228, -0.880, 0.162, ...]
- Cosine similarity ≈ 0.95 → très proches
- "Pizza margherita" → vecteur éloigné
- Espace vectoriel = représentation du sens



Évolution des Embeddings : Word2Vec → BERT

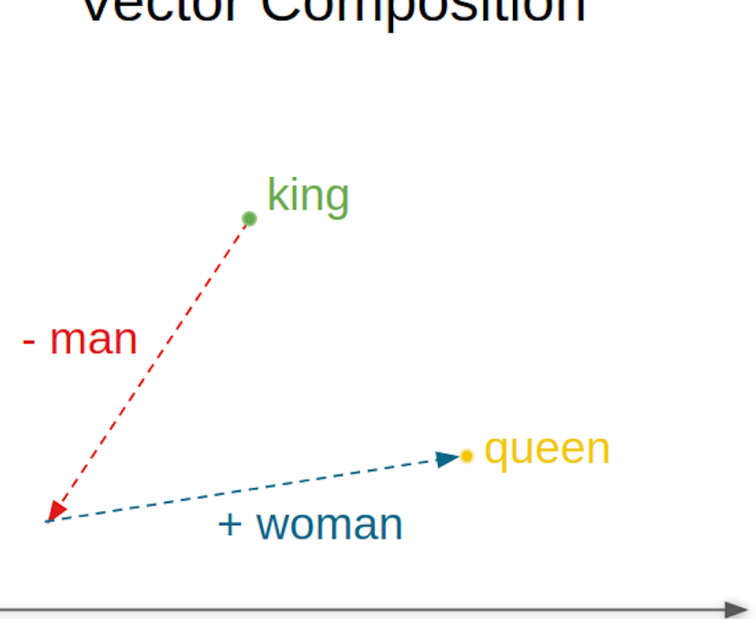
Word2Vec & GloVe (2013-2014) — Statiques

- Word2Vec Skip-gram : $\text{roi} - \text{homme} + \text{femme} \approx \text{reine}$
- GloVe : factorisation de co-occurrences globales
- fastText : sous-mots → mots rares mieux traités
- Vecteurs statiques : un seul vecteur par mot
- Limite : polysémie ignorée

BERT (2018) — Contextuels & Bidirectionnels

- "bank" (fleuve) $\neq$ "bank" (finance) → vecteurs différents
- Architecture Transformer 12 couches, 768D
- Pré-entraîné : Masked LM + Next Sentence Prediction
- Limite RAG : encodage requête+doc ensemble = lent
- → SBERT résout en séparant encodage (bi-encoder)

Vector Composition



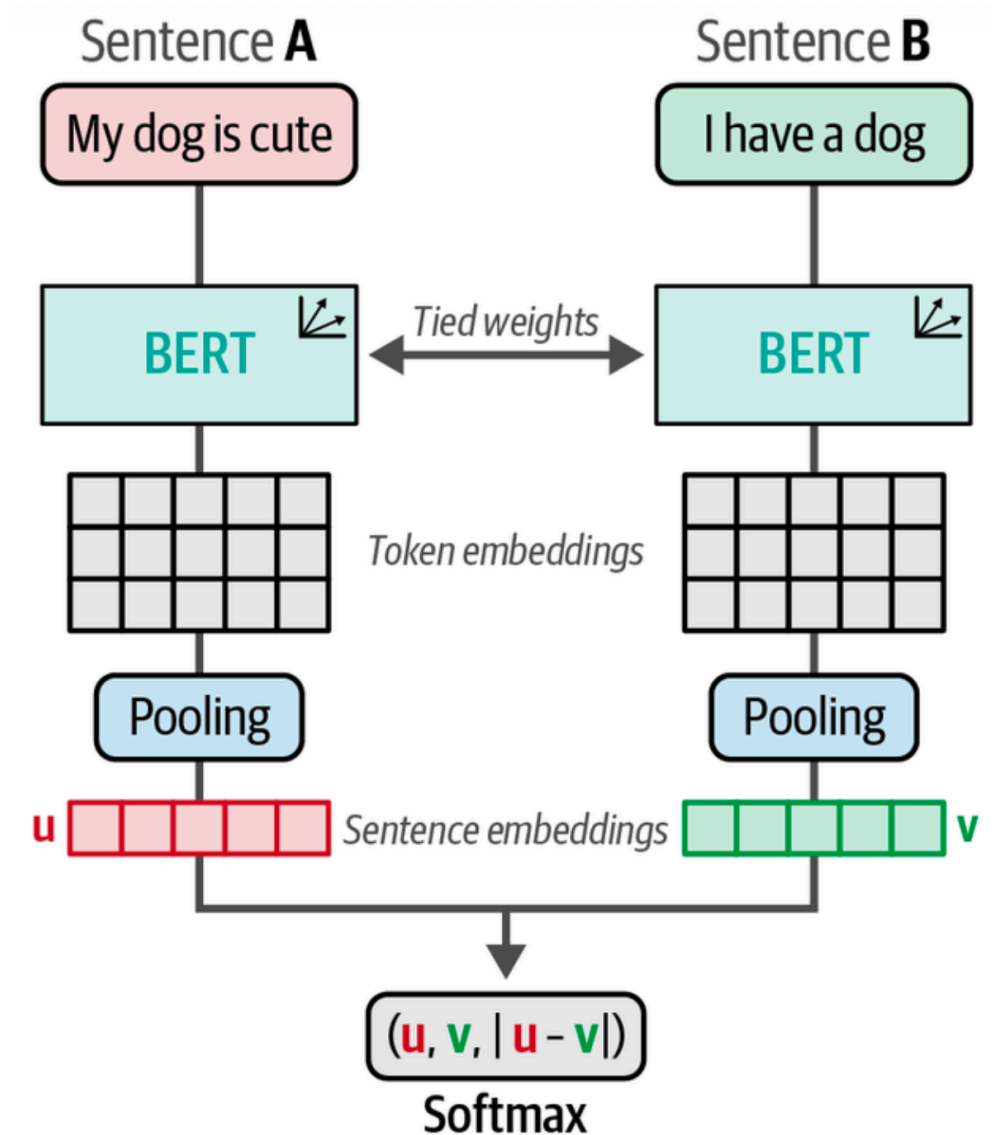
Sentence-BERT — Embeddings de Phrases

Architecture Bi-Encoder (Siamese)

- Requête et document encodés séparément
- Même modèle SBERT pour les deux
- $\text{Score} = \cos(v_{\text{requête}}, v_{\text{doc}})$
- Très rapide : embeddings pré-calculables
- all-MiniLM-L6-v2 (384D) — recommandé

Entraînement & Avantages

- Triplet Loss : doc similaire + doc négatif
- Sépare sémantiquement phrases proches vs éloignées
- 100x plus rapide que BERT cross-encoder
- Standard actuel pour RAG dense retrieval
- all-mpnet-base-v2 (768D) — meilleure qualité



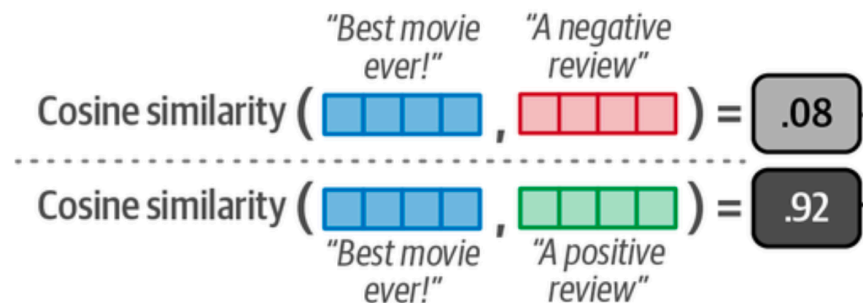
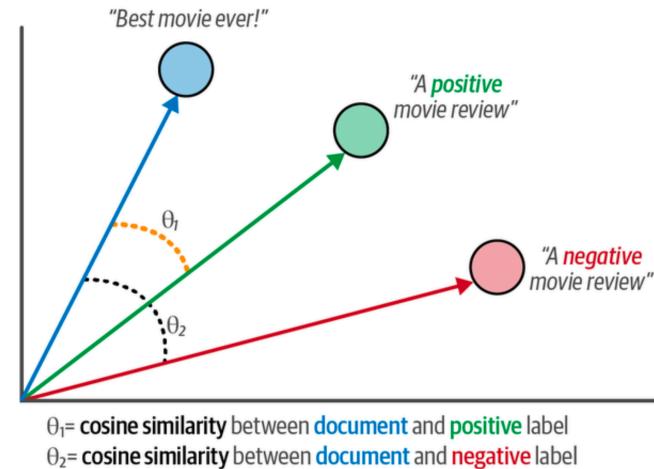
Similarité Cosinus & Métriques

Similarité Cosinus — Standard

- $\text{sim}(A,B) = (A \cdot B) / (||A|| \times ||B||)$
- Résultat : entre -1 et 1 (normalisé : 0 à 1)
- Mesure l'angle entre vecteurs (pas la magnitude)
- Invariant à la longueur du vecteur
- Recommandé pour semantic search

Comparaison des Métriques

- L2 (Euclidienne) : distance géométrique absolue
- Dot Product : rapide GPU, sans normalisation
- Cosine : intuitif, standard dans la littérature
- FAISS : supporte les trois métriques nativement
- Règle pratique : cosine pour qualité, dot pour vitesse



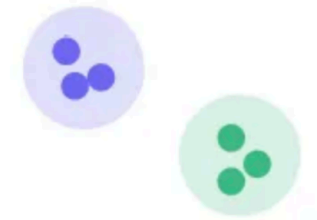
Approximate Nearest Neighbor (ANN)

Problème de Scalabilité

- Brute-force : $O(N \times D)$ trop lent à grande échelle
- 1M docs $\times$ 384D = 384M opérations par requête
- $\rightarrow$ Algorithmes ANN pour trade-off vitesse/précision

HNSW vs IVF vs PQ

- HNSW : graphe de proximité multi-niveaux, $O(\log N)$
- HNSW : recall 99%, 1-5ms, RAM-intensif
- IVF : partition en K clusters, moins de RAM
- IVF+PQ : compression 32x, bon pour >100M docs
- PQ : divise vecteur en m sous-espaces (uint8)



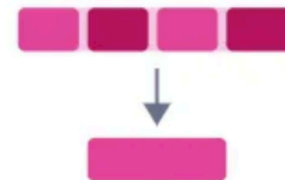
IVF

Clusters



HNSW

Graph



PQ

Compress

Bases de Données Vectorielles

Pourquoi une VectorDB ?

- SQL/NoSQL non optimisés pour similarité vectorielle
- Stockage + indexation + filtrage métadonnées
- Requêtes hybrides : vectoriel + filtres SQL
- Gestion des updates et deltas
- Scalabilité horizontale pour production

Comparatif des Systèmes

- Chroma : prototype, simple, local (dev)
- FAISS : bibliothèque locale, recherche offline (Meta)
- Qdrant : self-hosted, Rust, très performant
- Pinecone : cloud géré, excellent scalabilité
- Weaviate : graphe + vectors, hybride natif

Système	Algo ANN	Scale	Use Case
Chroma	HNSW	Proto / Dev	Local, dev
FAISS	IVF / HNSW	Offline	Recherche locale (Meta)
Qdrant	HNSW (Rust)	★ Très bonne	Self-hosted RAG prod
Pinecone	Propriétaire	★ Excellente	SaaS cloud
Weaviate	HNSW	Bonne	Graph + vectors
→ Qdrant recommandé pour self-hosted, Pinecone pour cloud géré			

MTEB Benchmark & Évaluation

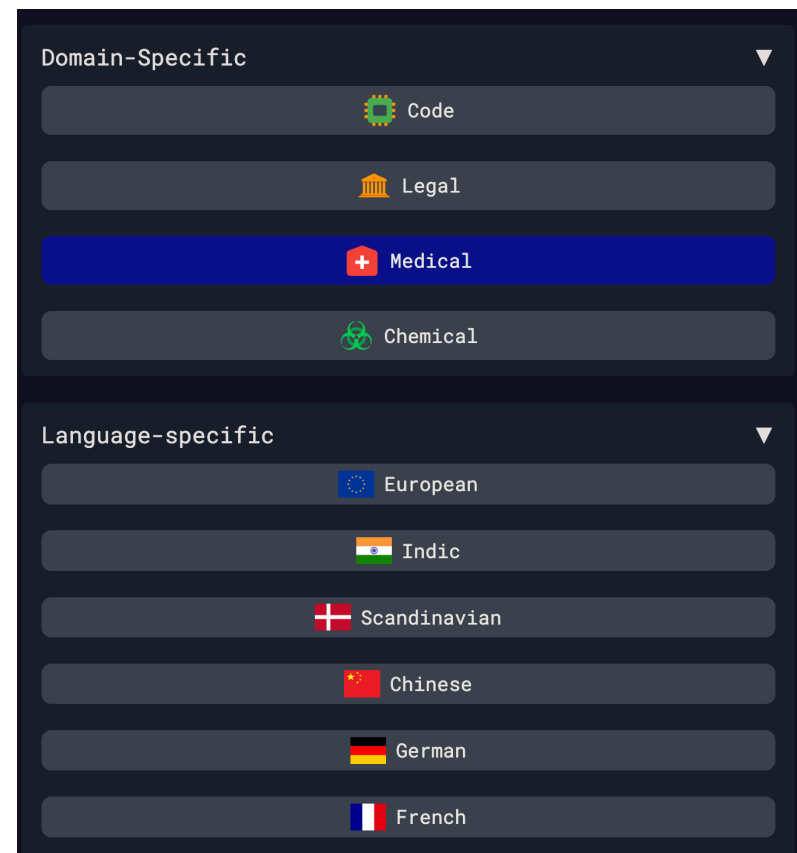
MTEB — Massive Text Embedding Benchmark

- Benchmark communautaire HuggingFace (2023)
- 56 tâches, 100+ langues, 100+ modèles
- Tâches : retrieval, clustering, classification, STS
- Score moyen sur toutes les tâches

Leaders du Classement - 2024 (ça évolue très vite...)

- E5-large-v2 (Microsoft) : excellent général
- BGE-M3 (Alibaba) : multilingue très performant
- all-MiniLM-L6-v2 : léger et efficace
- nomic-embed-text : open-source, très compétitif

• <https://huggingface.co/spaces/mteb/leaderboard>



2

Retrieval-Augmented Generation

Augmenter les LLMs avec données externes

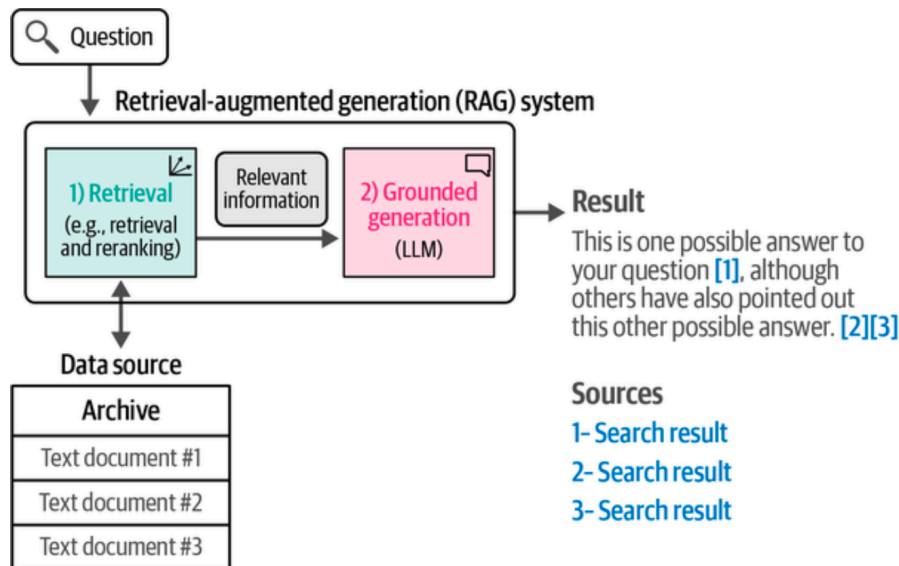
Limitations des LLMs — Motivation pour RAG

3 Limites Fondamentales des LLMs

- 1. Hallucinations : invente des faits avec assurance
- Ex : citations de papers inexistants, dates fausses
- Taux hallucination sans RAG : 15-30%
- 2. Knowledge Cutoff : données figées à une date
- Ex : GPT-4 cut-off avril 2023, ne connaît pas le reste

Autres Limites

- 3. Données Privées : corpus interne invisible
- Fine-tuning = coûteux, risqué, temps
- RAG : ancre la réponse dans des documents réels
- RAG : injecte infos récentes sans réentraînement
- RAG : accès contrôlé sans exposition des données



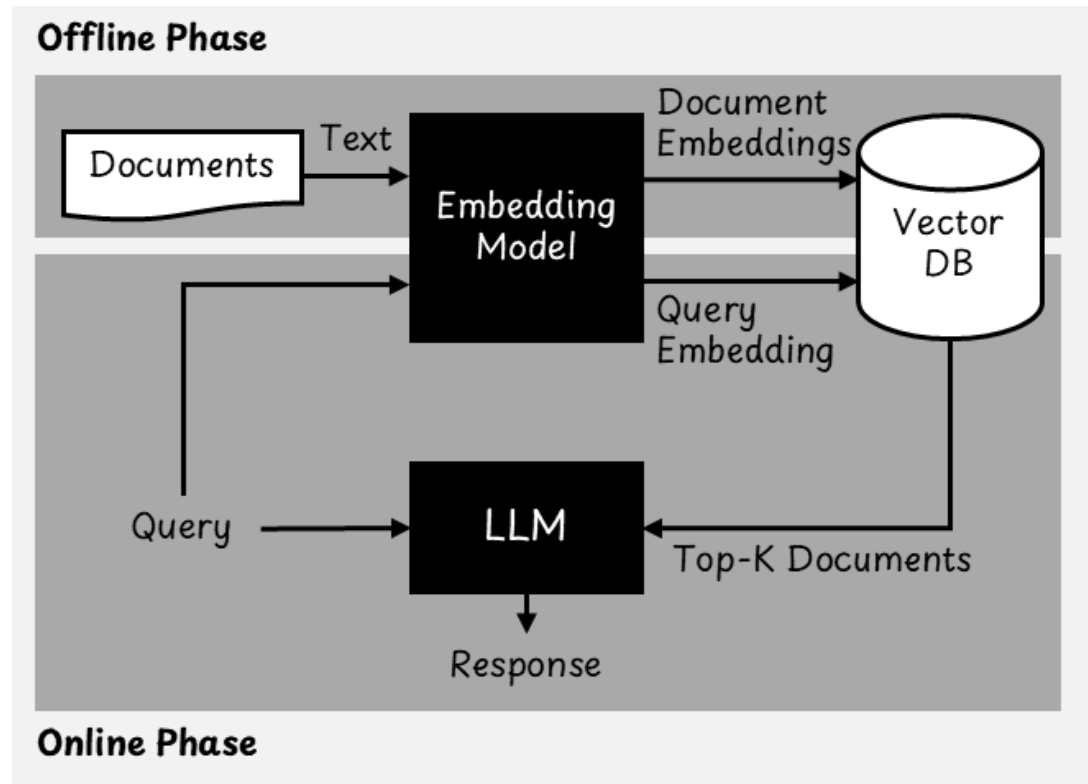
Architecture RAG Détaillée

Phase Offline (une seule fois)

- Docs → Parsing (pdfplumber, pymupdf)
- → Chunking (RecursiveCharacterSplitter)
- → Embedding (SBERT batch processing)
- → Indexation (Qdrant/FAISS avec HNSW)
- Durée : heures pour un grand corpus

Phase Online (chaque requête)

- Query → Embed (même modèle SBERT)
- → ANN Search ($O(\log N)$ lookup)
- → Reranking (cross-encoder top-10)
- → Prompt augmenté + LLM (GPT-4/Claude)
- Latence cible : <1 seconde end-to-end



Phase Offline — Ingestion & Chunking

Ingestion des Documents

- PDF : pdfplumber (texte) + pytesseract (OCR scannés)
- Formats variés : DOCX, CSV, HTML, unstructured.io
- Conserver métadonnées : source, date, page, offset
- Delta update : ne réembedder que le contenu nouveau
- Tables → markdown ou JSON pour préserver la structure

Stratégies de Chunking

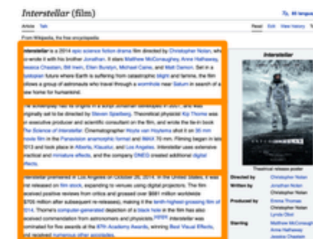
- Fixed-size : 512 tokens, overlap 50-100 tokens
- Recursive : découpe paragraphe → phrase → mot
- Semantic : regroupe par cohérence sémantique
- Trop petit → perd le contexte ; Trop grand → bruit
- → Tester avec RAGAS sur votre corpus réel

One vector per document



Document vector

Chunk document into multiple chunks



Chunk 1 vector
Chunk 2 vector
Chunk 3 vector

Each sentence is a chunk



Chunk 1 vector
Chunk 2 vector
Chunk 3 vector
...
Chunk 15 vector

Each paragraph is a chunk



Chunk 1 vector
Chunk 2 vector
Chunk 3 vector

Overlapping window of sentences



Chunk 1 vector
Chunk 2 vector
Chunk 3 vector

Sparse vs Dense vs Hybrid Retrieval

Comparatif des Approches

- Sparse (BM25) : 1ms, 65% accuracy, exact keywords
- Dense (SBERT) : 50ms, 85% accuracy, sémantique
- Hybrid : 55ms, 91% accuracy ← RECOMMANDÉ
- RRF (Reciprocal Rank Fusion) pour fusionner
- Pondération : $0.3 \times \text{BM25} + 0.7 \times \text{Dense}$

Quand Utiliser Quoi ?

- BM25 seul : corpus technique, termes exacts
- Dense seul : requêtes naturelles et paraphrases
- Hybrid : cas général de production (défaut)
- Termes rares, codes produits → BM25 essentiel
- Questions sémantiques ouvertes → Dense essentiel

Approche	Latence	Accuracy	Avantage
BM25 (Sparse)	~1ms ⚡	65%	Exact keywords très rapide
Dense (SBERT)	~50ms	85%	Sémantique paraphrases
Hybrid (BM25+Dense)	~55ms	91% ★	Meilleur des deux
Pondération recommandée : $0.3 \times \text{BM25} + 0.7 \times \text{Dense}$ (RRF)			
Gain Hybrid vs Dense seul : +4 à 6% accuracy sur BEIR benchmark			

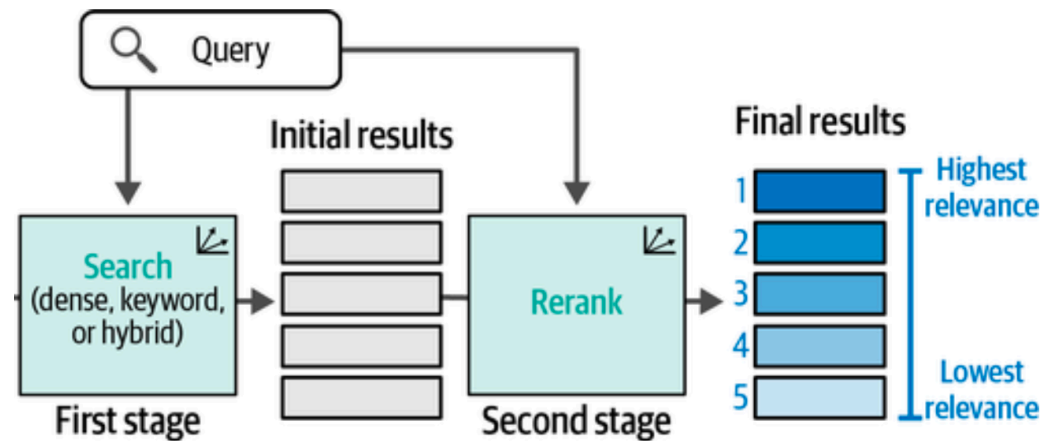
Reranking avec Cross-Encoders

Pourquoi Reranker ?

- Dense retrieval top-100 : rapide mais approximatif
- Cross-encoder : analyse paire (requête, doc) ensemble
- Attention croisée = compréhension plus fine
- +10-20% accuracy sur top-10 final
- Compromis : 10-100 docs/sec (vs 1000s bi-encoder)

Pipeline Bi-Encoder + Cross-Encoder

- Étape 1 : Dense retrieval → top-100 candidats
- Étape 2 : Cross-encoder rescoring top-100 → top-10
- Modèle : cross-encoder/ms-marco-MiniLM-L-6-v2
- mmarco (multilingue) pour corpus français
- Latence ajoutée : ~100-200ms pour 100 docs



Prompt Augmenté & Génération LLM

Anatomie du Prompt RAG

- System prompt : rôle + instructions de comportement
- Contexte : [1] doc1 [2] doc2 ... [k] dok (2000-3000 tokens)
- Question utilisateur + instruction de citation [1][2]
- Docs pertinents en premier, source (titre, page) incluse
- Si contexte insuffisant : "Je ne sais pas" obligatoire

Génération & Faithfulness

- GPT-4o, Claude Sonnet : meilleure qualité ; Mistral/Llama3 : local
- Temperature 0.1-0.3 pour réponses factuelles
- Faithfulness : réponse ancrée dans les docs (pas de complétion libre)
- Citer [1][2] = traçabilité et audit de la réponse
- Évaluer RAGAS Faithfulness score : cible > 0.85

```
from langchain import PromptTemplate

# Create a prompt template
template = """<|user|>
Relevant information:
{context}

Provide a concise answer the following question using the
relevant information provided above:
{question}<|end|>
<|assistant|>"""
prompt = PromptTemplate(
    template=template,
    input_variables=["context", "question"]
)

from langchain.chains import RetrievalQA

# RAG pipeline
rag = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type='stuff',
    retriever=db.as_retriever(),
    chain_type_kwargs={
        "prompt": prompt
    },
    verbose=True
)
```

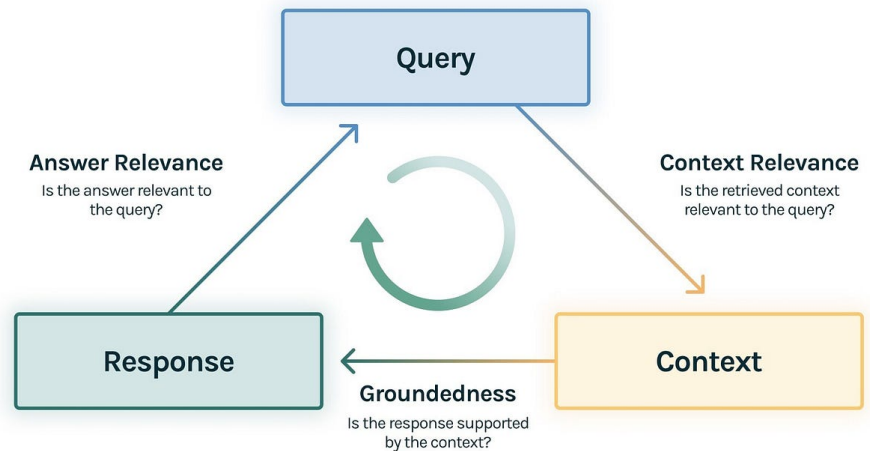
Évaluation RAG — Framework RAGAS

4 Dimensions d'Évaluation

- Faithfulness : réponse ancrée dans docs ? (cible >0.85)
- Answer Relevance : répond à la question ? (cible >0.80)
- Context Relevance : docs récupérés utiles ? (cible >0.75)
- Context Recall : infos nécessaires trouvées ? (cible >0.80)
- RAGAS Score = moyenne des 4 métriques

Comment Mesurer

- LLM-as-judge : automatique avec dataset de test
- Créer 50-200 paires (question, réponse attendue)
- pip install ragas → évaluation en quelques lignes
- Identifier le maillon faible (retrieval ou génération?)
- Itérer : tester chunking, k, modèle embedding



3

Étude de Cas & Multimodal

Implémentation pratique

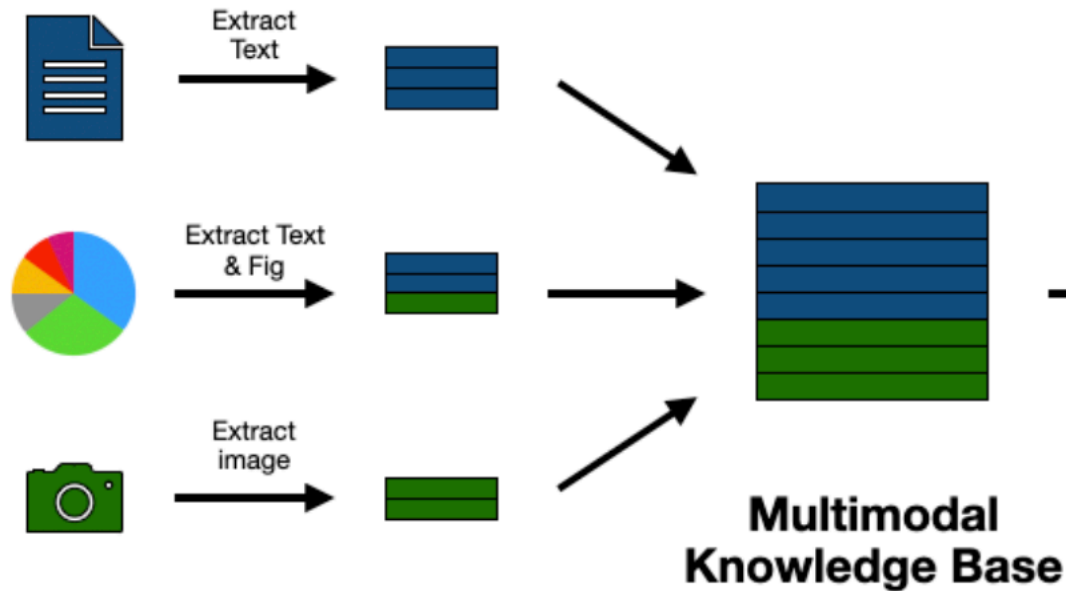
Motivation — Au-Delà du Texte

Limites du RAG Texte

- 30-40% de l'information est non-textuelle
- Images, diagrammes, graphiques, tableaux
- RAG texte = information incomplète et partielle
- Médecine : scanners + rapports médicaux liés
- Finance : graphiques + texte narratif liés

Pourquoi Multimodal RAG ?

- Documents riches : PDF complexes, articles, manuels
- Contenus imbriqués : texte + équations + figures
- Un seul espace vectoriel pour tous les types
- Requête texte → trouve images pertinentes
- Meilleure compréhension globale du document



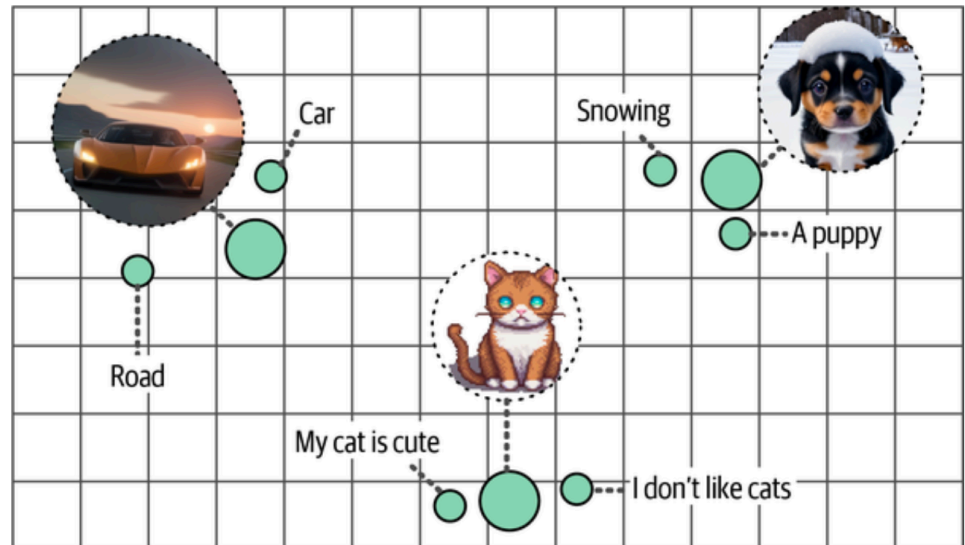
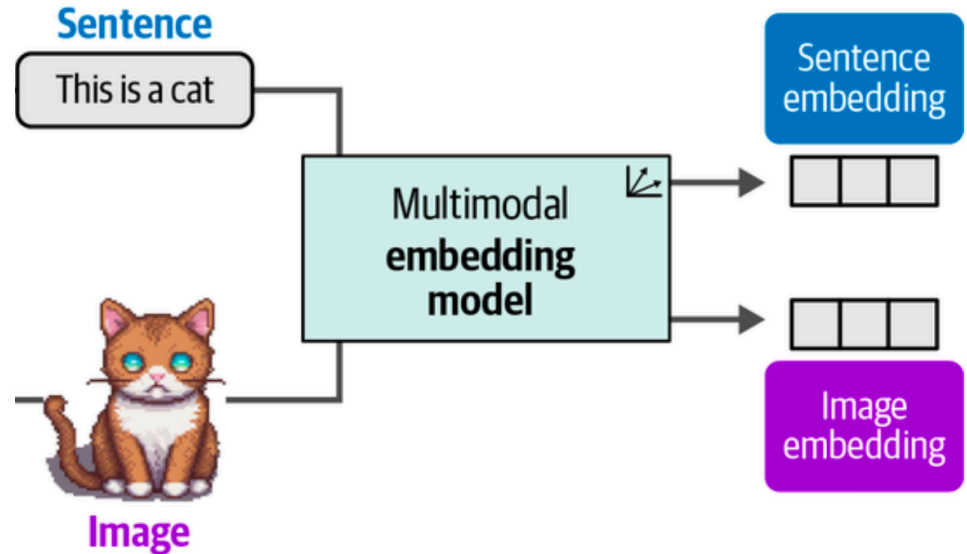
CLIP — Embeddings Visuels & Textuels

CLIP (OpenAI, 2021)

- Contrastive Language-Image Pre-Training
- Espace joint : images ET texte dans même espace vectoriel
- Image query → trouve textes pertinents (et vice versa)
- Entraîné sur 400M paires (image, caption) — LAION-400M
- Zero-shot : pas d'exemples d'entraînement nécessaires

Architectures

- Image Encoder : ViT-L/14, découpe en patches 14×14px
- Text Encoder : Transformer 12 couches (GPT-like)
- Sortie : vecteur 512D ou 768D pour les deux
- $\cos(v_image, v_text)$ = score de pertinence
- Alternative moderne : SigLIP, BLIP-2, Flamingo



Cas d'usages Industriels

Santé

- Q&A sur guidelines cliniques et protocoles
- Assistance au diagnostic avec imagerie médicale
- Recherche dans PubMed (~35M articles)
- Conformité HIPAA / RGPD stricte obligatoire
- Exemples : OpenEvidence, Glass Health

Finance & Droit

- Finance : Q&A rapports annuels 10-K, compliance
- Finance : Bloomberg GPT, FinGPT en production
- Droit : analyse contrats, recherche jurisprudentielle
- Droit : conformité RGPD, NIS2, DORA automatisée
- Droit : Harvey AI, Clio, LexisNexis + RAG

Secteur	Use Case	Outils	Défi clé
 Santé	Q&A guidelines cliniques, aide au diagnostic	OpenEvidence Glass Health	Conformité HIPAA/RGPD
 Finance	Rapports 10-K, compliance réglementaire	Bloomberg GPT FinGPT	Précision des chiffres critique
 Droit	Analyse contrats, recherche jurisprudentielle	Harvey AI Clio, LexisNexis	Responsabilité des erreurs
 Éducation	Assistant Q&A sur corpus académique	Qdrant + SBERT + Ollama	Volume et mise à jour
 Support	FAQ produit, documentation technique	Zendesk AI Intercom	Fraîcheur des infos

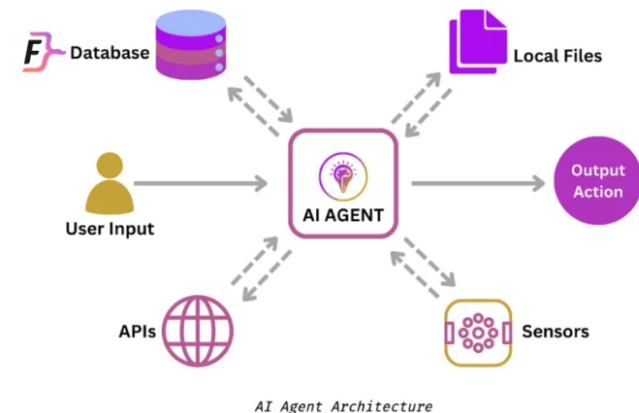
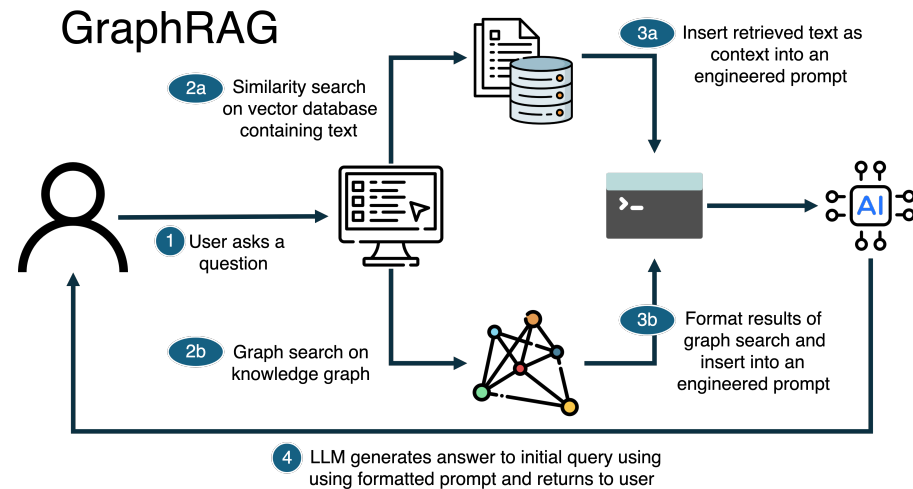
Challenges & Perspectives Futures

Problèmes ouverts actuels

- Long-context retrieval : Traiter de longs documents (>100K tokens) efficacement
- Multi-hop reasoning : raisonner sur 3+ documents
- Temporal reasoning : données changeantes dans le temps
- Continual learning : adapter sans tout réindexer
- Explainability : pourquoi ce document a été choisi ?

Prochaines frontières

- 4D RAG : temps + espace + modalités + graphes
- GraphRAG (Microsoft) : raisonnement sur graphes
- VideoRAG : indexation vidéo et audio
- ColPali : late interaction pour PDFs complets
- Agentic RAG : agents qui planifient le retrieval



Synthèse Générale — Points Clés

Recherche Sémantique

- SBERT/E5 = standard embedding 2024
- ANN indexing (HNSW) indispensable >10K docs
- Hybrid dense+BM25 = meilleur recall général
- MTEB leaderboard = référence comparaison modèles

RAG & Multimodal

- Réduit hallucinations de 60-80% en production
- Hybrid retrieval + reranking = pipeline optimal
- RAGAS framework pour mesurer et itérer
- Stack recommandé : Qdrant + SBERT + Ollama
- CLIP pour espace joint texte/image unifié

Références Bibliographiques

- [1] Karpukhin et al. (2020): Dense Passage Retrieval (DPR)
- [2] Lewis et al. (2020): RAG for Knowledge-Intensive NLP Tasks
- [3] Reimers & Gurevych (2019): Sentence-BERT
- [4] Radford et al. (2021): CLIP — Learning Transferable Visual Models
- [5] Devlin et al. (2018): BERT — Pre-training of Deep Bidirectional Transformers
- [6] Mikolov et al. (2013): Efficient Estimation of Word Representations (Word2Vec)
- [7] Nogueira et al. (2020): Passage Re-ranking with BERT
- [8] Es et al. (2023): RAGAS — Automated Evaluation of RAG
- [9] Gao et al. (2023): Retrieval-Augmented Generation for LLMs — A Survey
- [10] Izacard & Grave (2021): Leveraging Passage Retrieval with Generative Models